



Extending Harbinger – Change Points, Motifs & Custom Evaluation

What is a Change Point?

A change point signifies a moment in a time series where the underlying statistical properties, such as slope, mean, or variance, undergo an abrupt shift. Identifying these points is crucial for understanding system dynamics and predicting future behavior.



Identifying Shifts

Detects abrupt changes in statistical properties (e.g., mean, variance, slope) within time series data.



Segmented Regression

Utilizes joinpoint regression to fit piecewise linear models, precisely locating trend breakpoints.



Harbinger's Approach

Implements a 3-method contract: constructor, `fit()`, and `detect()` for seamless integration.

Common use cases include detecting trend reversals in sensor data, economic indicators, or clinical measurements, enabling timely interventions or insights.

 Reference: Muggeo, V.M.R. (2003). Estimating regression models with unknown break-points. *Statistics in Medicine*, 22(19), 3055–3071.

Custom Detector: Joinpoint Regression

Goal: detect points where a linear trend changes direction or slope, using `segmented::segmented()` as the estimation engine.

```
library(daltoolbox)
library(harbinger)
# install.packages("segmented")

hcp_joinpoint_custom <- function(npsi = 1) {
  obj <- harbinger()
  obj$npsi <- npsi
  class(obj) <- append(
    "hcp_joinpoint_custom", class(obj))
  obj
}

fit.hcp_joinpoint_custom <-
function(obj, data, ...) {
  x <- seq_along(data)
  base_model <- stats::lm(data ~ x)
  obj$model <- segmented::segmented(
    base_model,
    seg.Z = ~ x,
    npsi = obj$npsi)
  obj
}
```

```
detect.hcp_joinpoint_custom <-
function(obj, serie, ...) {
  obj <- obj$har_store_refs(obj, serie)
  if (is.null(obj$model))
    obj <- fit(obj, obj$serie)

  cp <- rep(FALSE, length(obj$serie))
  psi <- round(obj$model$psi[, "Est."])
  psi <- psi[psi >= 1 &
    psi <= length(cp)]
  cp[psi] <- TRUE

  obj$har_restore_refs(obj,
    change_points = cp)
}
```

Step by Step

- Constructor stores `npsi` (number of joinpoints)
- `fit()`: fit a base linear model with `lm(data ~ x)`
- Refine with `segmented()` to estimate breakpoints
- `detect()`: extract estimated positions from `$psi[, "Est."]`
- Round and validate indices within series bounds
- Mark positions as `change_points = TRUE`

📌 **Key concept:** Joinpoint regression connects change detection with regression interpretation — the breakpoint is where one straight line ends and another begins.

Fitting, Evaluating, and Visualizing the Joinpoint

The joinpoint detector reuses Harbinger's standard evaluation and plot workflow after converting the estimated breakpoint into the common detection table.

```
data(examples_changepoints)
dataset <- examples_changepoints$simple

model <- hcp_joinpoint_custom(npsi = 1)
model <- fit(model, dataset$serie)

## Warning: essentially perfect fit:
## summary may be unreliable

detection <- detect(model, dataset$serie)

# Avaliar
evaluation <- evaluate(model,
  detection$event,
  dataset$event)
evaluation$confMatrix
```

```
## event
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Visualizar
har_plot(model,
  dataset$serie,
  detection,
  dataset$event)
```

Step by Step

- Load `examples_changepoints$simple`
- Instantiate with `npsi = 1` (one joinpoint)
- Call `fit()` explicitly before `detect()`
- The "perfect fit" warning is expected for simple synthetic series
- Confusion matrix: 1 TP, 0 FP, 0 FN — perfect detection
- Plot to visually confirm the breakpoint

📌 **Key concept:** Fit structural model → convert estimate to detection table → reuse native evaluation and plot. This is the standard change-point extension pattern.

What is a Motif?

Motif detection focuses on identifying recurring subsequence patterns within time series data. Unlike anomaly detection, which seeks out the unusual, motif detection uncovers the most common and representative "signatures" or "shapes" that appear repeatedly.



What is a Motif?

Recurring subsequence patterns, identifying typical behaviors rather than anomalies.

Common applications include identifying recurring behaviors in ECG signals, detecting typical cycles in industrial machinery, or recognizing seasonal patterns in sensor data.

Reference: Sardá-Espinosa, A. (2019). Time-series clustering in R using the dtwclust package. The R Journal, 11(1), 22–43.



Why DTW?

Dynamic Time Warping (DTW) enables elastic alignment, recognizing similar patterns despite speed or phase shifts.



How Harbinger Wraps It

Leverages a 3-method contract (constructor, fit(), detect()), where fit() clusters and detect() selects representative instances.

Motif Detector: DTW Clustering – Constructor and Fit

Goal: discover motifs as families of similar subsequences via DTW clustering, instead of nearest-neighbor search through Matrix Profile.

```
library(daltoolbox)
library(harbinger)
# install.packages("dtwclust")

hmo_dtw_cluster_custom <-
  function(w = 20, centers = 3,
           min_cluster_size = 3) {
    obj <- harbinger()
    obj$w <- w
    obj$centers <- centers
    obj$min_cluster_size <- min_cluster_size
    class(obj) <- append(
      "hmo_dtw_cluster_custom", class(obj))
    obj
  }
```

```
fit.hmo_dtw_cluster_custom <-
  function(obj, data, ...) {
    windows <- tspredit::ts_data(data, obj$w)
    subsequences <- lapply(
      seq_len(nrow(windows)),
      function(i) as.numeric(windows[i, ]))

    obj$model <- dtwclust::tsclust(
      series = subsequences,
      type = "partitional",
      k = obj$centers,
      distance = "dtw_basic",
      centroid = "dba",
      seed = 1L,
      trace = FALSE)
    obj
  }
```

Step by Step

- Parameters: `w` (window size), `centers` (clusters), `min_cluster_size`
- `fit()`: extract sliding windows with `ts_data()`
- Convert each row into a list of numeric vectors
- Cluster with `tsclust()`: DTW distance, DBA centroid
- Store clustering model in `obj$model`

❏ **Key concept:** DTW allows elastic alignment — two similar patterns at different speeds are recognized as the same motif.

Motif Detector: `detect()` Method and Execution

The `detect()` method selects populous clusters and keeps non-overlapping representatives to prevent a single motif family from dominating the plot.

```
detect.hmo_dtw_cluster_custom <-  
function(obj, serie, ...) {  
  n <- length(serie)  
  non_na <- which(!is.na(serie))  
  serie_c <- stats::na.omit(serie)  
  if (is.null(obj$model))  
    obj <- fit(obj, serie_c)  
  
  cluster_id <- obj$model@cluster  
  cluster_sizes <- table(cluster_id)  
  selected <- names(  
    cluster_sizes[cluster_sizes >=  
      obj$min_cluster_size])  
  
  out <- data.frame(  
    event = rep(FALSE, length(serie_c)),  
    seq = rep(NA, length(serie_c)))
```

```
seq_counter <- 1  
for (cl in selected) {  
  positions <- which(  
    cluster_id == as.integer(cl))  
  sel <- integer(0); last <- -Inf  
  for (pos in positions) {  
    if ((pos - last) >= obj$w) {  
      sel <- c(sel, pos); last <- pos  
    }  
  }  
  out$event[sel] <- TRUE  
  out$seq[sel] <- as.character(seq_counter)  
  seq_counter <- seq_counter + 1  
}  
# montar detection e retornar ...  
}  
  
# Uso  
model <- hmo_dtw_cluster_custom(  
  w=15, centers=3, min_cluster_size=3)  
model <- fit(model, dataset$serie)  
detection <- detect(model, dataset$serie)  
har_plot(model, dataset$serie,  
  detection, dataset$event)
```

Step by Step

- Get cluster assignment: `obj$model@cluster`
- Filter clusters with size $\geq$ `min_cluster_size`
- For each selected cluster, choose non-overlapping representatives (distance $\geq w$)
- Mark positions as `event = TRUE` with sequence ID
- Build detection `data.frame` with `idx`, `event`, `type`, `seq`, `seqlen`

📌 **Key concept:** Recurring clusters = motif families. Non-overlapping representatives ensure the plot remains readable and informative.

Pointwise vs. Range-Aware Evaluation

Evaluating anomaly detection models, particularly for time series, requires nuanced approaches. Standard pointwise metrics often fall short when dealing with anomalies that span multiple data points or occur within a time window.

Pointwise Evaluation

Treats each timestamp independently. A detection is only considered correct if it precisely matches the ground truth at that specific point. This can be unfair for collective or sequence anomalies where a slightly misaligned but overlapping detection is entirely missed.

Range-Aware Evaluation

Introduced by Tatbul et al. (2018), this approach gives partial credit to detections that overlap the true anomaly interval. The `alpha` parameter balances the existence reward (if any overlap exists) with the overlap reward (how much overlap there is).

In Harbinger, custom evaluators follow a simple 2-method contract: `constructor()` and `evaluate()`. The `evaluate()` method receives detection and event vectors and returns performance metrics like precision, recall, and F1 score.

Use pointwise metrics for discrete point anomalies. Opt for range-aware evaluation when dealing with collective or sequence anomalies that extend over a duration, ensuring a fairer assessment of model performance.

Reference: Tatbul, N. et al. (2018). Precision and recall for time series. Advances in Neural Information Processing Systems (NeurIPS), 31.

Custom Evaluation: Pointwise vs. Range-Aware

The range-aware evaluator gives partial credit to detections that overlap the true interval, unlike pointwise evaluation which can be unfair for collective anomalies.

```
# Exemplo didático com intervalos conhecidos
```

```
n <- 100
```

```
event <- rep(FALSE, n)
```

```
detection <- rep(FALSE, n)
```

```
# Dois intervalos verdadeiros
```

```
event[20:30] <- TRUE
```

```
event[60:75] <- TRUE
```

```
# Uma detecção parcial, uma boa, uma falsa
```

```
detection[18:27] <- TRUE
```

```
detection[64:78] <- TRUE
```

```
detection[88:92] <- TRUE
```

```
# Avaliação pontual (dura)
```

```
hard_eval <- evaluate(har_eval(),  
                      detection, event)
```

```
hard_eval$confMatrix
```

```
## event
```

```
## detection TRUE FALSE
```

```
## TRUE 20 10
```

```
## FALSE 7 63
```

```
# Avaliação range-aware (crédito parcial)
```

```
range_eval <- evaluate(  
  har_eval_range_custom(alpha = 0.5),  
  detection, event)
```

```
## $precision [1] 0.5333333
```

```
## $recall [1] 0.8693182
```

```
## $F1 [1] 0.6610856
```

```
## $alpha [1] 0.5
```

Step by Step

- Create event and detection vectors with known intervals
- Evaluate with `har_eval()`: strict pointwise metric
- Evaluate with `har_eval_range_custom(alpha=0.5)`: partial credit
- Compare: pointwise ignores partial overlap; range-aware rewards it
- Precision 0.53, Recall 0.87, F1 0.66 — better reflects actual quality

❏ **Key concept:** The choice of metric must reflect the anomaly type. Sequence anomalies require range-aware metrics — pointwise metrics can be misleading.

Summary of Extension Points

Each extension respects a minimal contract. The smaller the contract, the easier it is to integrate external libraries.

Change Point

Contract: constructor + `fit()` + `detect()`

Example: `hcp_joinpoint_custom` with `segmented::segmented()` — detects where a linear trend changes slope via joinpoint regression.

Motif

Contract: constructor + `fit()` + `detect()`

Example: `hmo_dtw_cluster_custom` with `dtwclust::tsclust()` — discovers recurring subsequence families via DTW clustering.

Evaluation

Contract: constructor + `evaluate()`

Example: `har_eval_range_custom` with interval-based precision/recall — gives partial credit to detections that overlap the true anomaly interval.

Additional Sources

Explore the resources below to dive deeper into Harbinger and time series event detection.



CRAN Package

cran.r-project.org/web/packages/harbinger



Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series



DAL Toolbox

cran.r-project.org/web/packages/daltoolbox



Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.

References

Ogasawara, E. et al. (2025). Event Detection in Time Series. Springer. — Primary reference for the Harbinger framework, covering change point, motif, and evaluation extensions.

Muggeo, V.M.R. (2003). Estimating regression models with unknown break-points. *Statistics in Medicine*, 22(19), 3055–3071. — Reference for the segmented R package used in `hcp_joinpoint_custom`.

Sardá-Espinosa, A. (2019). Time-series clustering in R using the dtwclust package. *The R Journal*, 11(1), 22–43. — Reference for `dtwclust::tsclust()` used in `hmo_dtw_cluster_custom`.

Tatbul, N. et al. (2018). Precision and recall for time series. *Advances in Neural Information Processing Systems (NeurIPS)*, 31. — Reference for range-aware evaluation metrics used in `har_eval_range_custom`.